

Data Extraction

All the raw data was downloaded as CSV manually from

<https://dados.turismo.gov.br/dataset/chegada-de-turistas-internacionais>

Updating the Data

1. Download the new CSV from the link above and save it in `data/raw/`. Any file name ending in `.csv` is read.
2. The ministry publishes the current year as **cumulative** files (for example `base_chegadas_2026_acum_08_agosto.csv`, January–August). When a newer cumulative file of the same year arrives, **delete the older one**. Each year must come from exactly one file; otherwise its months would be counted twice. The extraction below stops with an error if two files contain the same year.
3. Re-run the notebooks in order. The train/test split (notebook 2) always holds out the **last 12 months** of the data, and every later notebook reads its dates from the split, so no date needs to be edited by hand.

If a new file uses a column name that is not in `COLUMN_MAPPING`, the extraction stops and lists the file's columns. Add the new name to `COLUMN_MAPPING` and re-run.

```
In [1]: import os

import pandas as pd
from tqdm.auto import tqdm

ROOT_PATH = os.path.dirname(os.getcwd())
DATA_FOLDER_PATH = os.path.join(ROOT_PATH, 'data')
```

Data Import

Understanding how column names change over time

```
In [2]: last_adopted_column_pattern = ''

for file in sorted(os.listdir(os.path.join(DATA_FOLDER_PATH, 'raw'))):
    if not file.endswith('.csv'):
        continue

    df = pd.read_csv(
        os.path.join(DATA_FOLDER_PATH, 'raw', file),
        encoding='latin-1',
        low_memory=False,
        sep=';',
    )
```

```

if last_adopted_column_pattern != str(df.columns):
    last_adopted_column_pattern = str(df.columns)

print(f'File: {file}')
print(last_adopted_column_pattern)
print('\n')

```

File: base_chegadas_2026_acum_08_agosto.csv

```

Index(['Via_de_acesso', 'UF', 'nome_pais_correto', 'mes', 'ano', 'Chegadas'], dtype='object')

```

File: chegadas_1989.csv

```

Index(['Continente', 'Ordem continente', 'País', 'Ordem país', 'UF',
      'Ordem UF', 'Via de acesso', 'Ordem via de acesso', 'ano', 'Mês',
      'Ordem mês', 'Chegadas'],
      dtype='object')

```

File: chegadas_2000.csv

```

Index(['Continente', 'Ordem continente', 'País', 'Ordem país', 'UF',
      'Ordem UF', 'Via de acesso', 'Ordem via de acesso', 'Ano', 'Mês',
      'Ordem mês', 'Chegadas'],
      dtype='object')

```

File: chegadas_2015.csv

```

Index(['Continente', 'Ordem continente', 'País', 'Ordem país', 'UF',
      'Ordem UF', 'Via de acesso', 'Ordem via de acesso', 'ano', 'Mês',
      'Ordem mês', 'Chegadas'],
      dtype='object')

```

File: chegadas_2016.csv

```

Index(['Continente', 'cod continente', 'País', 'cod país', 'UF', 'cod uf',
      'Via', 'cod via', 'ano', 'Mês', 'cod mes', 'Chegadas'],
      dtype='object')

```

File: chegadas_2025.csv

```

Index(['Via_de_acesso', 'UF', 'nome_pais_correto', 'mes', 'ano', 'Chegadas'], dtype='object')

```

```

In [3]: COLUMN_MAPPING = {
        'access_route': ['Via de acesso', 'Via', 'Via_de_acesso'],
        'state': ['UF'],
        'origin_country': ['País', 'nome_pais_correto'],
        'month': ['Mês', 'mes'],
        'year': ['ano', 'Ano'],
        'arrival_count': ['Chegadas'],
    }

SELECTED_COLUMNS = list(COLUMN_MAPPING.keys())

```

```

def standardize_columns(df: pd.DataFrame, file_name: str) -> pd.DataFrame:
    rename_map = {
        variant: target
        for target, variants in COLUMN_MAPPING.items()
        for variant in variants
        if variant in df.columns
    }
    df = df.rename(columns=rename_map)
    missing = [column for column in SELECTED_COLUMNS if column not in df.columns]
    if missing:
        raise KeyError(
            f'{file_name}: no source column for {missing}. '
            f'Columns in the file: {list(df.columns)}. '
            'Add the new column name to COLUMN_MAPPING.'
        )
    return df[SELECTED_COLUMNS]

```

Unifying the data

```

In [4]: dfs = []
files_by_year = {}

csv_files = sorted(
    file
    for file in os.listdir(os.path.join(DATA_FOLDER_PATH, 'raw'))
    if file.endswith('.csv')
)

for arrival_file in tqdm(csv_files):
    try:
        df = pd.read_csv(
            os.path.join(DATA_FOLDER_PATH, 'raw', arrival_file),
            encoding='utf-8',
            low_memory=False,
            sep=';',
        )

    except UnicodeDecodeError:
        df = pd.read_csv(
            os.path.join(DATA_FOLDER_PATH, 'raw', arrival_file),
            encoding='latin-1',
            low_memory=False,
            sep=';',
        )

    df = standardize_columns(df, arrival_file)
    for year in df['year'].dropna().astype(int).unique():
        files_by_year.setdefault(int(year), []).append(arrival_file)
    dfs.append(df)

# each year must come from exactly one file (cumulative releases of the same year
# would otherwise count the overlapping months twice)
duplicated_years = {

```

```

        year: files for year, files in files_by_year.items() if len(files) > 1
    }
    if duplicated_years:
        raise ValueError(
            'The same year appears in more than one CSV. Keep only the most recent '
            f'file of each year in data/raw: {duplicated_years}'
        )

df = pd.concat(dfs).reset_index(drop=True)

df.head()

```

0%| | 0/38 [00:00<?, ?it/s]

Out[4]:

	access_route	state	origin_country	month	year	arrival_count
0	Aérea	Bahia	Afeganistão	Janeiro	2026	1.0
1	Aérea	São Paulo	Afeganistão	Janeiro	2026	4.0
2	Terrestre	Paraná	Afeganistão	Janeiro	2026	1.0
3	Terrestre	Rio Grande do Sul	Afeganistão	Janeiro	2026	1.0
4	Marítima	Rio de Janeiro	Afeganistão	Janeiro	2026	3.0

Exporting

In [5]:

```

df.to_parquet(
    os.path.join(DATA_FOLDER_PATH, 'raw', 'raw_arrivals.parquet'), index=False
)

```